

Strings, f-strings, Dictionaries, Sorting & JSON-style Data

Detailed reference notes for everything covered in class — with examples, outputs and key rules.

Contents

- 1. String Validation Methods — isalpha, isdigit, isalnum, startswith, endswith
- 2. f-strings — substitution, expressions, number formatting
- 3. Dictionaries — creation, access, every important method, looping, nesting, comprehension
- 4. Sorting — sorted(), reverse=True, the key=lambda pattern
- 5. JSON-style Data — list of dictionaries: filter, extract, group
- 6. Quick Reference — one-page method cheat sheet

1. String Validation Methods

Python strings come with a family of **validation methods** that answer yes/no questions about the content of a string. They all return **True** or **False**, which makes them perfect inside `if` conditions when cleaning and validating user input.

» The character-check family

These three methods inspect **every single character** of the string:

Method	Returns True when...	Common trap
<code>s.isalpha()</code>	every character is a letter (A–Z, a–z)	a space or a digit anywhere makes it False
<code>s.isdigit()</code>	every character is a digit (0–9)	a decimal point, minus sign or space makes it False
<code>s.isalnum()</code>	every character is a letter or a digit	symbols like #, _, @, space make it False

```
print("hello ".isalpha())    # the trailing space is not a letter
print("1234.5".isdigit())    # '.' is not a digit
print("hello#123".isalnum()) # '#' is neither a letter nor a digit

print("hello".isalpha())     # clean versions pass
print("12345".isdigit())
print("hello123".isalnum())
```

OUTPUT

```
False
False
False
True
True
True
```

Note — One wrong character anywhere makes the whole answer **False**. An **empty string** also returns False for all three — there is no character to satisfy the check.

» startswith() and endswith()

These check only the **beginning** or the **end** of a string. Both are **case-sensitive** and are the standard tools for prefix/suffix validation: hashtags (#), phone codes (+91), URLs (https://), file types (.pdf), email domains (.com).

```
print("#VST".startswith("#VS"))      # True
print("Google.com".endswith(".com"))  # True
print("Google.com".endswith(".COM"))  # False -> case matters
```

Note — Both methods can also accept a **tuple** of options: `name.endswith((".jpg", ".png"))` returns True if the string ends with any one of them.

» Worked example 1 — cleaning + validating a name

Problem: A user enters " rahul KUMAR sharma ". Print it properly as Rahul Kumar Sharma, and print Invalid name if it contains anything other than alphabets and spaces.

Pipeline: `strip()` trims the ends → `title()` capitalises each word → `split()` (no argument) removes the extra inner spaces and gives a word list to validate → `" ".join()` rebuilds the clean name with single spaces.

```
name = "  rahul KUMAR sharma  "

name = name.strip().title()      # trim ends + capitalise each word
words = name.split()             # also eats the extra inner spaces

valid = True
for w in words:
    if not w.isalpha():          # every word: alphabets only
        valid = False
        break

if valid:
    print(" ".join(words))
else:
    print("Invalid name")
```

OUTPUT

Rahul Kumar Sharma

» Worked example 2 — messy string of marks

Problem: Marks arrive as one messy string " 45, 88 ,72, 91 , 60 ". Convert to a list of integers, then print the total and average.

`split(",")` cuts at every comma, but each piece still carries spaces — so every piece is `strip()`-ed before `int()`, all inside one list comprehension.

```
x = " 45, 88 ,72, 91 , 60 "

parts = x.split(",")              # ['45', ' 88 ', '72', ' 91 ', ' 60 ']
marks = [int(p.strip()) for p in parts]  # clean each piece, convert to int

print("Total    :", sum(marks))
print("Average :", sum(marks) / len(marks))
```

OUTPUT

Total : 356
Average : 71.2

2. f-strings

An **f-string** is created by putting `f` before the opening quote. Anything inside `{ }` is treated as Python code, evaluated, and its result is placed into the string. It replaces the clumsy comma-separated `print ( )` style.

» Basic substitution

```
name = "Harsh"
age = 21

# old style: comma-separated pieces (awkward spacing)
print("Name of student is ", name, " and age is ", age)

# f-string: one readable string
print(f"Name of student is {name} and age is {age}")
```

OUTPUT

```
Name of student is  Harsh  and age is  21
Name of student is Harsh and age is 21
```

» Expressions inside { }

The braces can hold **any expression** — calculations, method calls, function calls, even indexing. Python evaluates first, then substitutes.

```
a = 5
b = 7
print(f"The sum of {a} and {b} is {a + b}") # calculation
print(f"The name is {name.upper()}")      # method call
```

OUTPUT

```
The sum of 5 and 7 is 12
The name is HARSH
```

» Number formatting — the format spec

After the value, a colon starts a **format spec**. The one used in class was `:.2f` — *fixed-point number, 2 digits after the decimal* (with rounding). Ideal for prices, averages and percentages.

```
price = 3456.789
print(f"The price is {price:.2f}") # 2 decimal places (rounded)
print(f"The price is {price:.1f}") # 1 decimal place
```

OUTPUT

```
The price is 3456.79
The price is 3456.8
```

Spec	Meaning	Example on 3456.789
<code>:.2f</code>	2 digits after the decimal	3456.79
<code>:.0f</code>	no decimals (rounds)	3457

Spec	Meaning	Example on 3456.789
: ,	thousands separator (good to know)	3,456.789

3. Dictionaries

A dictionary stores data as `key: value` pairs inside curly braces. It is Python's lookup table — you find values by key, not by position.

- Keys must be **unique**. Assigning to an existing key **overwrites** its value.
- Keys must be **immutable** (string, number, tuple). Lists and dicts cannot be keys. Values can be anything.
- Since Python 3.7 a dict remembers **insertion order**.

» Access — `d[key]` vs `d.get(key)`

Situation	<code>d[key]</code>	<code>d.get(key, default)</code>
Key exists	returns the value	returns the value
Key missing	crashes with <code>KeyError</code>	returns <i>default</i> (or <code>None</code>) — no crash

```
d = {"aman": 23, "harsh": 27, "Aditi": 29, "Mohit": 12}

# print(d["aman1"])           # KeyError: 'aman1' -> crash
print(d.get("aman1", "key missing")) # safe: prints the default
```

OUTPUT
key missing

» `setdefault(key, default)` — get + insert if missing

- Key **exists** → returns its value, dict unchanged.
- Key **missing** → inserts `key: default` into the dict **and** returns default.

Unlike `get()`, this method can **modify** the dictionary — which is exactly why it is the classic tool for grouping and counting patterns.

```
print(d.setdefault("aman1", 0)) # missing -> inserted with 0, returns 0
print(d)                       # 'aman1': 0 is now inside
```

OUTPUT
0
{'aman': 23, 'harsh': 27, 'Aditi': 29, 'Mohit': 12, 'aman1': 0}

» Views — `keys()`, `values()`, `items()`

These return **live view objects**: if the dict changes, the views update automatically. `items()` yields `(key, value)` tuples and is the workhorse for looping and sorting.

```
print(d.keys())
print(d.values())
print(d.items())
```

OUTPUT

```
dict_keys(['aman', 'harsh', 'Aditi', 'Mohit', 'aman1'])
dict_values([23, 27, 29, 12, 0])
dict_items([('aman', 23), ('harsh', 27), ('Aditi', 29), ('Mohit', 12), ('aman1', 0)])
```

» Adding and updating

- `d[new_key] = value` — adds one pair (or overwrites an existing key).
- `d.update(other)` — merges many pairs at once; overlapping keys take the new value.

```
d = {"aman": 23, "harsh": 27, "Aditi": 29, "Mohit": 12}

d["ankit"] = 50          # single pair
d.update({"a": 3, "b": 6}) # many pairs at once
print(d)
```

OUTPUT

```
{'aman': 23, 'harsh': 27, 'Aditi': 29, 'Mohit': 12, 'ankit': 50, 'a': 3, 'b': 6}
```

» Removing — `pop()`, `popitem()`, `del`, `clear()`

Tool	What it does
<code>d.pop(key)</code>	removes that key and returns its value ; <code>KeyError</code> if missing, unless a default is given: <code>d.pop(key, default)</code>
<code>d.popitem()</code>	removes and returns the last inserted (key, value) pair
<code>del d[key]</code>	removes that key; returns nothing
<code>del d</code>	deletes the entire variable — using <code>d</code> afterwards raises <code>NameError</code>
<code>d.clear()</code>	empties the dict in place — <code>d</code> still exists as <code>{}</code>

```
print(d.pop("a"))          # removes 'a', returns its value
print(d.popitem())         # removes & returns the LAST pair -> ('b', 6)

temp = {"x": 1}
del temp                  # whole variable gone
# print(temp)             # NameError: name 'temp' is not defined

d2 = {"aman": 23, "harsh": 27}
d2.clear()                # empty in place; variable survives
print(d2)
```

OUTPUT

```
3
('b', 6)
{}
```

Note — `del d` and `d.clear()` are very different: **`del`** destroys the variable itself, while **`clear()`** only empties the container and keeps the variable usable.

» Empty dict vs empty set

Curly braces are shared by two types. `{}` is always an empty **dict**; an empty **set** must be written `set()`. Braces *with* plain values (no colons), like `{1, 4, 5}`, make a set.

```
d = {}
print(type(d))    # <class 'dict'>

st = set()
print(type(st))   # <class 'set'>

st = {1, 4, 5}    # values without colons -> set
st.clear()
print(st)         # empty set prints as set()
```

OUTPUT

```
<class 'dict'>
<class 'set'>
set()
```

» dict.fromkeys(keys, value)

Builds a new dict where **every** key starts with the **same** value — handy for initialising score sheets, counters, and defaults.

```
marks = dict.fromkeys(["math", "english", "physics"], 0)
print(marks)
```

OUTPUT

```
{'math': 0, 'english': 0, 'physics': 0}
```

» Looping patterns + membership

- for k in d: — loops over the **keys**.
- for k, v in d.items(): — unpacks key and value together (most common pattern).
- "aman" in d — membership checks the **keys**, never the values.

```
d = {"aman": 23, "harsh": 27, "Aditi": 29, "Mohit": 12}

for k, v in d.items():
    print(k, v)

print("aman" in d)    # True -> checks keys
print(23 in d)        # False -> values are not checked
```

OUTPUT

```
aman 23
harsh 27
Aditi 29
Mohit 12
True
False
```

» Nested dictionaries

A value can itself be a dictionary. Reach inner data by **chaining** keys: first the outer key, then the inner one.

```
college = {
    "CS": {"hod": "Dr. Mehta", "students": 120},
    "AI": {"hod": "Dr. Rao", "students": 90},
}

for k in college.keys():
    print(college[k]["hod"]) # outer key -> inner key
```

OUTPUT

```
Dr. Mehta
Dr. Rao
```

» Dict comprehension

Same idea as a list comprehension, but each element is a key: value pair.

```
d = {i: i**2 for i in range(1, 11)}
print(d)
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
```

» sorted(d.items()) — sorting by key

sorted() applied to d.items() returns a **list of (key, value) tuples** arranged by key (alphabetical for strings). Sorting by *value* needs key=lambda — next section.

```
d = {"aman": 23, "harsh": 27, "aditi": 29, "mohit": 12}
d1 = sorted(d.items())
print(d1)
```

OUTPUT

```
[('aditi', 29), ('aman', 23), ('harsh', 27), ('mohit', 12)]
```

4. Sorting with key=lambda

» sorted() basics

- sorted(iterable) returns a **new sorted list**; the original is untouched.
- reverse=True flips the order (descending).
- It works on any iterable: lists, tuples, strings, dict views.

```
nums = [45, 12, 91, 30]
print(sorted(nums))           # ascending
print(sorted(nums, reverse=True)) # descending
print(nums)                   # original unchanged
```

OUTPUT

```
[12, 30, 45, 91]
[91, 45, 30, 12]
[45, 12, 91, 30]
```

» The key= argument

key= takes a **function**. Python calls that function on every element and sorts by the **returned value**, not the element itself. Lambda gives a quick one-line function:

Note — `lambda x: x[1]` reads as “take one input `x`, return `x[1]`” — here, the second item of a tuple.

```
words = ["banana", "fig", "apple", "kiwi"]
print(sorted(words, key=len))          # sort by LENGTH

pairs = [("aman", 23), ("harsh", 27), ("aditi", 29), ("mohit", 12)]
print(sorted(pairs, key=lambda x: x[1])) # sort tuples by 2nd item
print(sorted(pairs, key=lambda x: x[1], reverse=True))
```

OUTPUT

```
['fig', 'kiwi', 'apple', 'banana']
[('mohit', 12), ('aman', 23), ('harsh', 27), ('aditi', 29)]
[('aditi', 29), ('harsh', 27), ('aman', 23), ('mohit', 12)]
```

» Sorting a dict by its values

Combine the two ideas: `d.items()` supplies (key, value) tuples, and `key=lambda x: x[1]` sorts them by the value part.

```
d = {"aman": 23, "harsh": 27, "aditi": 29, "mohit": 12}

by_value = sorted(d.items(), key=lambda x: x[1], reverse=True)
print(by_value)
print(dict(by_value))  # back to a dict if needed
```

OUTPUT

```
[('aditi', 29), ('harsh', 27), ('aman', 23), ('mohit', 12)]
{'aditi': 29, 'harsh': 27, 'aman': 23, 'mohit': 12}
```

» Sorting a list of dicts

The same pattern scales directly to JSON-style records: pick the field inside the lambda.

```
students = [{"name": "Amit", "marks": 85}, {"name": "Bina", "marks": 72}]
print(sorted(students, key=lambda s: s["marks"], reverse=True))
```

OUTPUT

```
[{'name': 'Amit', 'marks': 85}, {'name': 'Bina', 'marks': 72}]
```

5. JSON-style Data (List of Dictionaries)

API and JSON data almost always arrives as a **list of records**, where each record is a **dict**. Mastering three patterns — **filter**, **extract**, **group** — lets you process any such dataset.


```

students = [
    {"name": "Amit", "age": 20, "marks": 85, "city": "Pune"},
    {"name": "Bina", "age": 22, "marks": 72, "city": "Mumbai"},
    {"name": "Chirag", "age": 21, "marks": 91, "city": "Pune"},
    {"name": "Deepa", "age": 23, "marks": 65, "city": "Delhi"},
    {"name": "Esha", "age": 20, "marks": 78, "city": "Mumbai"},
]

products = [
    {"pname": "Laptop", "price": 55000, "category": "Electronics", "stock": 8},
    {"pname": "Mouse", "price": 500, "category": "Electronics", "stock": 50},
    {"pname": "Notebook", "price": 60, "category": "Stationery", "stock": 200},
    {"pname": "Pen", "price": 10, "category": "Stationery", "stock": 500},
    {"pname": "Chair", "price": 3500, "category": "Furniture", "stock": 15},
]

```

» Pattern 1 — Filter

Loop through the records and keep only those matching a condition on one field.

```

for p in products:
    if p["price"] > 1000:
        print(p["pname"], "->", p["price"])

```

OUTPUT

```

Laptop -> 55000
Chair -> 3500

```

» Pattern 2 — Extract (with de-duplication)

Pull one field out of every record into a list. To avoid duplicates, add only when the value is not already present.

```

cats = []
for p in products:
    if p["category"] not in cats:
        cats.append(p["category"])
print(cats)

```

OUTPUT

```

['Electronics', 'Stationery', 'Furniture']

```

» Pattern 3 — Group

Build a dict of {group_key: [items]}. `.setdefault()` creates the empty list the first time a group appears, then every matching item is appended.

```

by_age = {}
for s in students:
    by_age.setdefault(s["age"], []).append(s["name"])
print(by_age)

# equivalent longer form:
# if s["age"] not in by_age:
#     by_age[s["age"]] = []
# by_age[s["age"]].append(s["name"])

```

OUTPUT

```
{20: ['Amit', 'Esha'], 22: ['Bina'], 21: ['Chirag'], 23: ['Deepa']}
```

6. Quick Reference

» Strings

Method	Purpose
<code>isalpha()</code> / <code>isdigit()</code> / <code>isalnum()</code>	all-letters / all-digits / letters+digits check
<code>startswith(p)</code> / <code>endswith(s)</code>	prefix / suffix check (case-sensitive)
<code>strip()</code> · <code>title()</code> · <code>lower()</code>	trim ends · capitalise words · lowercase
<code>split(sep)</code> / <code>" ".join(lst)</code>	string → list / list → string
<code>f"{x:.2f}"</code>	f-string with 2-decimal formatting

» Dictionaries

Method	Purpose
<code>d[k]</code> / <code>d.get(k, default)</code>	access (crash vs safe)
<code>d.setdefault(k, default)</code>	get + insert if missing (modifies dict)
<code>d.keys()</code> / <code>values()</code> / <code>items()</code>	live views; <code>items()</code> gives (k, v) tuples
<code>d[k] = v</code> / <code>d.update(other)</code>	add or overwrite pairs
<code>d.pop(k)</code> / <code>d.popitem()</code>	remove by key (returns value) / remove last pair
<code>del d[k]</code> / <code>del d</code> / <code>d.clear()</code>	delete key / delete variable / empty in place
<code>dict.fromkeys(keys, v)</code>	same starting value for every key
<code>{k: v for ...}</code>	dict comprehension

» Sorting

Idiom	Purpose
<code>sorted(x)</code> / <code>sorted(x, reverse=True)</code>	new list, ascending / descending
<code>sorted(x, key=len)</code>	sort by length
<code>sorted(d.items())</code>	dict sorted by key → list of tuples
<code>sorted(d.items(), key=lambda x: x[1])</code>	dict sorted by value
<code>sorted(records, key=lambda r: r["field"])</code>	list of dicts sorted by a field